NAME: ADEBIMPE ADEROJU M.

COURSE: CSE210- PROGRAMMING WITH FUNCTIONS

INSTRUCTURE'S NAME: JOHN READING

What is polymorphism and why is it important?

Polymorphism means "many forms" and allows objects of different classes to be treated as instances of a common base class while each respond to the same method call in its own specific way. When you call a method on a base class reference, C# automatically executes the correct overridden version from the actual object type at runtime. This creates flexible, extensible code where you write one loop or method that works with any derived class without knowing the exact type.

Key Benefit: Single interface, multiple behaviors. You can process collections of mixed goal types using one GetDisplayString() call per goal, and each goal automatically shows its unique format (checkbox for simple, progress for checklist, etc.). This eliminates massive if-else chains and makes adding new goal types trivial.

Application in Eternal Quest: My program lists all goals using a single loop through a List. Polymorphism ensures each goal displays correctly:

{

// Base class defines the interface public virtual string GetDisplayString() // ← Virtual method { return $"[ ] {_name} ({_description})"; }

// ChecklistGoal overrides with unique behavior public override string GetDisplayString() // ← Override + polymorphism! { string status = _isCompleted ? "[X]" : $"[{_completedCount}/{_totalRequired}]"; return $"{status} {_name} ({_description})"; }

// Program.cs - ONE loop handles ALL goal types for (int i = 0; i < _goals.Count; i++) { Console.WriteLine($"{i + 1}. {_goals[i].GetDisplayString()}"); // ← Magic happens here! }
How it works: When I call goal.GetDisplayString(), C# checks the actual object type at runtime:

SimpleGoal → shows [ ] or [X]

ChecklistGoal → shows [3/10] progress

EternalGoal → always shows [ ]

}

The loop doesn't know or care about specific types - it just calls the polymorphic method. Output becomes:

{

1. ☑ Run Marathon (1000 pts)
2. [3/10] Temple Attendance (50 pts + 500 bonus)
3. ☐ Read Scriptures (100 pts daily)

}

Result: Adding a new HabitGoal class requires one override, not changing the display loop. Polymorphism = future-proof code that scales effortlessly.